

## Program-8

**AIM:-decision tree classifier. Visualize the tree structure. Evaluate accuracy, precision, recall, and F1-score.**

### **Objective:**

To develop a program to load the Titanic dataset, split it into training and testing sets, train a Decision Tree classifier, visualize the tree structure, and evaluate the model using accuracy, precision, recall, and F1-score.

### **Theory:**

The Titanic dataset is used to implement a **supervised learning classification model**, where the target variable is **Survived (0 = No, 1 = Yes)**. The aim is to learn a mapping function from input features (such as *Pclass*, *Sex*, *Age*, *Fare*) to the output class.

Initially, **data preprocessing** is performed to improve data quality and model performance. This includes:

- **Handling missing values** using imputation techniques (mean/median for numerical features)
- **Encoding categorical variables** using methods like Label Encoding or One-Hot Encoding
- **Feature selection** to remove irrelevant or redundant attributes

The dataset is then divided into **training and testing sets** using a splitting method (e.g., 80:20 ratio). The training set is used to learn the model parameters, while the testing set is used to evaluate generalization.

A **Decision Tree Classifier** is applied, which is a non-parametric supervised learning algorithm. It constructs a tree by recursively partitioning the feature space. At each node, the best split is chosen based on impurity measures such as:

- **Gini Index:**  
Measures the probability of incorrect classification
- **Entropy (Information Gain):**  
Measures the reduction in randomness after the split

The objective is to minimize impurity and create pure leaf nodes containing a single class.

The tree grows until stopping criteria are met, such as:

- Maximum depth of the tree
- Minimum number of samples per node

After training, the model is used to **predict class labels** for the test dataset. The learned decision rules can be visualized as a tree structure, improving interpretability.

Model performance is evaluated using a **confusion matrix**, which includes:

- True Positive (TP), True Negative (TN)
- False Positive (FP), False Negative (FN)

From this, the following metrics are computed:

- **Accuracy** =  $(TP + TN) / (TP + TN + FP + FN)$
- **Precision** =  $TP / (TP + FP)$
- **Recall** =  $TP / (TP + FN)$
- **F1-Score** =  $2 \times (Precision \times Recall) / (Precision + Recall)$

These metrics provide a comprehensive evaluation of the classifier's performance, especially in handling class imbalance and prediction quality.

```
2]: # Import libraries
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
import matplotlib.pyplot as plt

# Load dataset
data = pd.read_csv("titanic.csv")

# Select only important columns (to keep it simple)
data = data[['Survived', 'Pclass', 'Sex', 'Age']]

# Convert 'Sex' to numbers (male=0, female=1)
data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})

# Fill missing Age values
data['Age'].fillna(data['Age'].mean(), inplace=True)

# Split data
X = data[['Pclass', 'Sex', 'Age']]
y = data['Survived']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

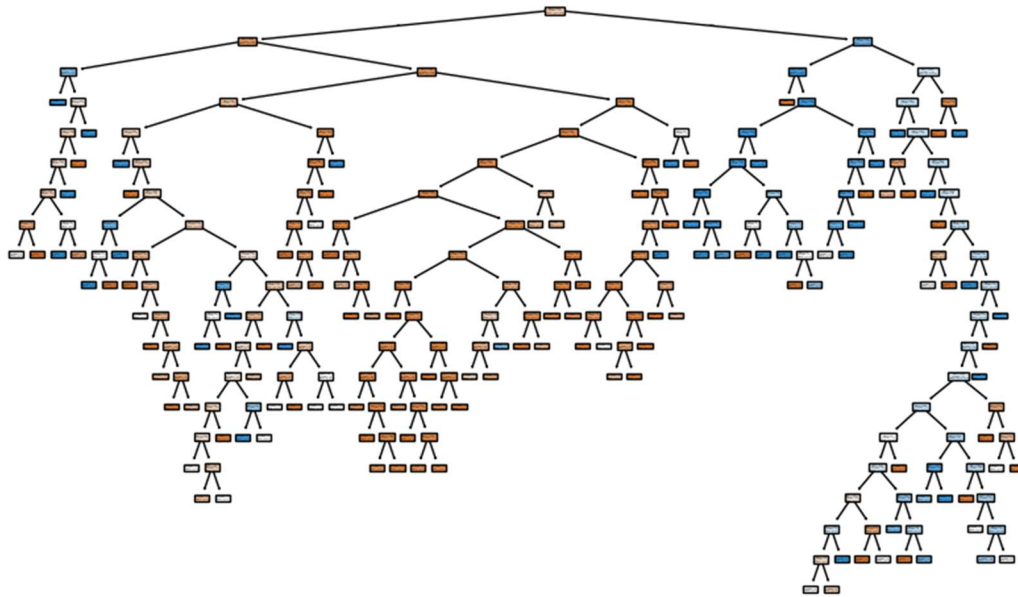
# Train model
model = DecisionTreeClassifier()
model.fit(X_train, y_train)

# Predict
y_pred = model.predict(X_test)

# Evaluate
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Precision:", precision_score(y_test, y_pred))
print("Recall:", recall_score(y_test, y_pred))
print("F1 Score:", f1_score(y_test, y_pred))

# Visualize tree
plt.figure(figsize=(10,6))
plot_tree(model, feature_names=X.columns, filled=True)
plt.show()
```

Accuracy: 0.7653631284916201  
Precision: 0.7037037037037037  
Recall: 0.59375  
F1 Score: 0.6440677966101694



## Result:

The Decision Tree model was successfully trained and tested on the Titanic dataset. The performance metrics such as accuracy, precision, recall, and F1-score were calculated, showing the effectiveness of the model.